

# Итоговый проект по дисциплине «Внедрение моделей машинного обучения»

## Название проекта

Разработка и внедрение сервиса прогнозирования дефолта по кредитным картам с контейнеризацией и A/B-тестированием

## Описание проекта

Цель проекта: разработать и внедрить в production-like-среду сервис машинного обучения для прогнозирования дефолта в виде масштабируемого веб-сервиса, готового к полному циклу от сохранения модели до организации A/B-тестирования.

Домен: финансы / кредитный скоринг.

Датасет: Default of Credit Card Clients Dataset с UCI Machine Learning Repository.

- Ссылка: [Default of Credit Card Clients Dataset](#).
- Описание: датасет содержит информацию о клиентах кредитной карты на Тайване с targetом default.payment.next.month (дефолт в следующем месяце). Включает демографические данные, историю платежей, суммы счетов.
- Актуальность: идеально подходит для демонстрации полного цикла внедрения ML-модели в production без излишней сложности в части построения модели.

## Задание к проекту

### Контекст

Вы — ML-инженер в технологическом отделе банка. Ваша задача — внедрить модель прогнозирования дефолта в виде масштабируемого веб-сервиса, готового к промышленной эксплуатации и тестированию новых версий.

### Требования к проекту

Часть 1. Подготовка модели к production и деплой

- Организация кода и контроль версий (Git):
  - Создайте структурированный репозиторий на GitHub (например, используя шаблон cookiecutter-data-science).
  - Код должен быть хорошо организован в модули (папки src/, tests/, notebooks/, data/, models/ и так далее).
- Сохранение и загрузка модели:
  - Обучите простую модель бинарной классификации (LogisticRegression, RandomForestClassifier или GradientBoostingClassifier) на предоставленном датасете.
  - Сохраните обученную модель с использованием pickle или joblib.
  - Напишите функцию загрузки модели для инференса.
- Веб-сервис на Flask:
  - Реализуйте веб-сервис с двумя эндпоинтами:
  - POST /predict — принимает JSON с признаками клиента, возвращает прогноз и вероятность.
  - GET /health — проверяет работоспособности сервиса.
  - Формат запроса/ответа должен быть чётко документирован.
- Дополнительно (обзорно):
  - В документации опишите, как можно преобразовать модель в формат ONNX-ML для оптимизации.
  - Объясните, для чего используются UWSGI + NGINX в production-среде.

Часть 2. Воспроизводимость и контейнеризация

- Воспроизводимость окружения:
  - Создайте requirements.txt со всеми зависимостями проекта.
  - Используйте virtualenv или venv для изоляции окружения.
- Docker-контейнеризация:
  - Напишите Dockerfile для упаковки веб-сервиса в контейнер.
  - Соберите Docker-образ и протестируйте его локально.
  - Загрузите образ в Docker Hub (публичный).
- Создание образа веб-сервиса:
  - Образ должен включать модель, код сервиса и все зависимости.
  - Контейнер должен запускать веб-сервис на определённом порту (например, 5000).

Часть 3. Сервисная архитектура и оркестрация

- Архитектура сервиса (концепт):
  - Монолит vs микросервисы.

Опишите, как в гипотетическом сценарии масштабирования можно было бы использовать брокер очередей (например, RabbitMQ). Для чего он мог бы пригодиться? Пример: для асинхронной обработки батч-предсказаний или логирования. Опционально: реализуйте такой брокер сообщений в своём проекте.
  - Логирование и мониторинг (обзорно).

Укажите в коде или документации, куда и в каком формате (JSON) логируются запросы и ответы API. Упомяните, как в production-системе эти логи могли бы собираться (например, ELK-стек) для мониторинга.
- Оркестрация (Docker Compose — опционально, бонус).

Создайте базовый файл docker-compose.yml:

  - который запускает ваш ML-сервис;
  - (задание повышенной сложности) добавляет сервис для имитации логов (например, простой ping) или визуализирует концепт подключения мониторинга.
- Обзор инструментов MLOps (концепт).

В документации кратко опишите назначение DVC (контроль версий данных) и MLflow (отслеживание экспериментов) в контексте жизненного цикла данной модели. Реализация не требуется.
- Бизнес-метрики.

Помимо технических метрик (F1, Precision), предложите одну-две бизнес-метрики, которые важны для заказчика. Например: снижение ожидаемых финансовых потерь или увеличение доли одобренных заявок при том же уровне риска. Опишите, как их можно было бы рассчитать на основе выходов модели.

Часть 4. Организация A/B-тестирования

- Концепция A/B-теста.

Предложите постановку A/B-теста для сравнения двух версий модели:

  - контрольной группы: текущей модели (v1);
  - тестовой группы: новой модели (v2).

Описание должно включать:

  - способ разделения трафика (например, случайное распределение 50/50);
  - продолжительность теста.
- Метрики и анализ:

Определите бизнес-метрики для оценки (минимум две):

  - основную метрику: F1-score для класса дефолта;
  - дополнительную метрику: Precision или Recall (с обоснованием выбора).

Опишите метод статистического анализа:

  - какой статистический тест будет использоваться (t-test, z-test);
  - как будут строиться доверительные интервалы;
  - каков критерий успешности теста.

Часть 5. Документация и воспроизводимость

- README.md:
  - Описание проекта и его целей.
  - Инструкция по запуску (локально и в Docker).
  - Примеры запросов к API (curl-команды).
  - Описание формата запросов и ответов.
  - Ссылка на Docker-образ в Docker Hub.
- Структура репозитория.

Чёткая организация файлов (отдельные папки для кода, моделей, конфигураций).

## Критерии оценки (50 баллов максимум)

### Инженерная реализация

| Критерии                   | Макс. баллов | Описание                                                |
|----------------------------|--------------|---------------------------------------------------------|
| Работоспособность API      | 8            | Эндпоинты /predict и /health работают, формат корректен |
| Качество Docker-реализации | 7            | Рабочий Dockerfile, образ собрался и запускается        |
| Воспроизводимость          | 4            | requirements.txt, инструкции по запуску                 |
| Сохранение/загрузка модели | 3            | Использование pickle/joblib                             |

### Сервисная архитектура и оркестрация

| Критерии                | Макс. баллов | Описание                                                                      |
|-------------------------|--------------|-------------------------------------------------------------------------------|
| Обоснование архитектуры | 8            | В документации дан внятный анализ «монолит vs микросервисы» для данного кейса |
| Концепты MLOps          | 5            | Описаны концепты RabbitMQ, логирования, DVC/MLflow, бизнес-метрики            |
| Docker Compose (бонус)  | +3           | Наличие рабочего docker-compose.yml                                           |

### A/B-тестирование

| Критерии                      | Макс. баллов | Описание                                                       |
|-------------------------------|--------------|----------------------------------------------------------------|
| Постановка и метрики          | 4            | Чёткий план A/B-теста с бизнес-метриками                       |
| Анализ и связь с архитектурой | 3            | Описание статистических методов и влияния на дизайн сервиса    |
| Практическая демонстрация     | 3            | Возможность протестировать две версии модели (v1/v2) через API |

### Оформление и документация

| Критерии              | Макс. баллов | Описание                               |
|-----------------------|--------------|----------------------------------------|
| Качество документации | 3            | Полный README с архитектурным разделом |
| Структура проекта     | 2            | Чистая организация репозитория         |

Максимальное количество баллов за весь проект: 50

## Пример решения (ключевые компоненты)

### 1. Рекомендуемая архитектура проекта

```
credit-card-a1-deployment/
├── app/
│   ├── init.py
│   ├── api.py # Flask-приложение
│   └── model_handler.py # Загрузка и использование модели
├── models/
│   ├── train_model.py # Скрипт обучения модели
│   ├── predict.py # Скрипт предсказания
│   └── model_v1.pkl # Сохраненная модель
├── tests/
│   └── test_api.py # Тесты API
├── docker/
│   └── Dockerfile # Конфигурация Docker
├── requirements.txt # Зависимости Python
├── docker-compose.yml # Опционально: для оркестрации
├── ab_test_plan.md # Документация по A/B-тесту
└── README.md # Основная документация
```

### 2. Пример Flask-приложения (app/api.py)

```
from flask import Flask, request, jsonify
import pickle
import numpy as np

app = Flask(__name__)

# Загрузка модели из pickle
with open('models/v1.pkl', 'rb') as f:
    model = pickle.load(f)

@app.route('/predict', methods=['POST'])
def predict():
    """Endpoint для предсказания дефолта"""
    try:
        data = request.get_json()
        features = preprocess_input(data)
        prediction = model.predict(features)
        probability = model.predict_proba(features)[0][1]

        return jsonify([
            'prediction': int(prediction[0]),
            'probability': float(probability),
            'model_version': 'v1'
        ])
    except Exception as e:
        return jsonify({'error': str(e)}), 400

@app.route('/health', methods=['GET'])
def health():
    """Проверка здоровья сервиса"""
    return jsonify({'status': 'healthy'}), 200

def preprocess_input(data):
    """Преобразование входных данных"""
    # Преобразование JSON в numpy array
    features = np.array([data[key] for key in sorted(data.keys())]).reshape(1, -1)
    return features

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=False)
```

### 3. Пример Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

# Копирование зависимостей
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Копирование кода и модели
COPY app/ ./app/
COPY models/ ./models/

# Запуск порта
EXPOSE 5000

# Запуск приложения
CMD ["python", "app/api.py"]
```

## Формат сдачи

Студент должен предоставить:

- Ссылку на GitHub-репозиторий, который содержит:
  - весы код проекта;
  - документацию (README.md);
  - файлы конфигурации.
- Инструкцию по запуску в README.md, включая:
  - установку зависимостей;
  - запуск сервиса локально;
  - сборку и запуск Docker-контейнера;
  - примеры запросов к API.
- Артефакты:
  - файл обученной модели (.pkl или .joblib);
  - requirements.txt;
  - Dockerfile;
  - план A/B-теста (отдельный файл или в README).
- Демонстрацию работы:
  - скриншоты или логи работы API;
  - пример curl-запроса с выводом;
  - ссылку на Docker-образ в Docker Hub.

## Ограничения и рекомендации

- Сложность модели: не требуется глубокий нейнинг или сложные ансамбли. Достаточно одной: правильно обученной модели.
- Инфраструктура:
  - Не требуется настройка NGINX/uWSGI в качестве обязательной части.
  - RabbitMQ и микросервисы могут быть описаны на концептуальном уровне.
  - Docker Compose — опционально для бонусных баллов.
- Фокус проекта: минимально достаточный production-like pipeline — работающий сервис, готовый к развёртыванию, с продуманным планом тестирования.
- Проверка: убедитесь, что проект можно полностью воспроизвести по инструкции, а Docker-образ доступен для скачивания.

Это задание охватывает все ключевые аспекты курса, делая акцент на практическое внедрение ML-моделей в production-среду с учётом реалистичных требований к промышленной эксплуатации.

Задание, оцениваемое ментором

### Внедрение моделей ML. Итоговый проект (PJ)

Домашнее задание проверит ментор и поставит оценку. Если возникнут сложности, обращайтесь в канал модуля. Желаем успехов! :)

Сдать домашнее задание

Сдать

← Предыдущий урок

Следующий урок →